PORTFOLIO PROJECT / UPDATED SPECIFICATION

SafeAct

Verified action execution for AI agents and integrations

> **Record the intent. Establish what happened. Repeat an action only when its connector contract makes that safe.**

SafeAct is a backend service and review application for external actions whose outcomes become uncertain after timeouts, worker crashes, duplicate delivery, or partial workflow completion. Callers include backend services, webhook consumers, and tool-using AI agents.

The first business scenario is a support agent issuing a refund. The fault occurs after the provider applies the refund but before the caller receives its response. SafeAct preserves the uncertainty, obtains available evidence, and prevents an unsupported retry.

What this revision changes

- Replaces lookup-then-retry assumptions with explicit provider guarantees and unresolved states.
- Separates operation identity, attempts, local worker ownership, and remote side-effect protection.
- Adds authenticated human review, tenant authorization, and evaluation of agent behavior.
- Uses stronger baselines, separate outcome metrics, and one real sandbox integration.
- Replaces calendar promises with evidence-based implementation phases.

Deliverable and positioning

Deliver a working API, durable workers, three controlled connectors, one real sandbox connector, an MCP interface, a bounded demo agent, a substantial React review console, and a replayable fault suite. The main evidence is for backend, infrastructure software, full-stack, and AI platform roles. Applied-agent claims require the agent evaluation on page 6.

Status: planned work. This document contains requirements and experiments, not measured results, certifications, a novel distributed-systems theorem, or a hiring guarantee.

02 Problem, boundaries, and credible contribution

A timeout is an observation about the caller, not proof that an external system did nothing. An action may have failed before delivery, committed without an acknowledgement, or remain in progress. The provider can also retry a webhook or expose eventually consistent lookups. These behaviors are documented in AWS and Stripe guidance [S1, S2].

User	Concrete need	Finished evidence
Backend engineer	Recover a timed-out action without inventing its outcome.	Attempt history, provider evidence, a justified decision, and reconciliation.
Support operator	Understand an unresolved refund and choose an allowed next step.	A permissioned review workflow with evidence and stale-decision protection.
AI engineer	Prevent a tool-using agent from repeating or approving restricted actions.	A guarded MCP tool interface and repeated task-level evaluations.

Scope

- Required: single-action execution; three mock connector profiles; one fixed, bounded multi-step scenario; verification; safe compensation; escalation; MCP; authenticated UI; experiments; observability; local reproducibility.
- Required external qualification: implement the payments connector against Stripe sandbox or another documented provider sandbox. Credentials belong to the implementer; production money and customer data are unnecessary.
- Excluded: a general workflow language, arbitrary code execution, universal schema repair, multi-region operation, production SSO, and claims of compatibility with every API.

Relationship to existing systems

Durable workflow engines can implement verification activities; idempotent APIs and reconciliation are established patterns. SafeAct packages explicit connector capabilities, uncertainty handling, evidence review, and evaluated agent access into one bounded application. A comparison must credit those patterns instead of claiming existing engines cannot implement them.

> **Claim scope: observed correctness within named connector contracts and tested faults. Unresolvable actions remain visibly unresolved; escalation is not successful business completion.**

03 Operation identity and connector contracts

Stable operation identity

Create an immutable operation_id before dispatch. Bind it to the authenticated tenant, permitted business action, target resource, and canonical payload hash. Retries reuse the same logical operation and provider idempotency key. Each network attempt gets a separate attempt_id. A caller presenting an existing key with a different payload receives a conflict; it does not silently reuse the previous result.

The server establishes or validates operation identity. An agent cannot bypass a duplicate guard simply by inventing a new key for the same authorized operation. Distinct authorized requests can still be distinct business actions: semantic deduplication of arbitrary human intent is outside the guarantee.

Capability	Contract that must be documented
Native idempotency	Key scope, retention period, parameter binding, concurrent-request behavior, and expiry semantics. Never assume a key works indefinitely.
Lookup	Accepted correlation fields, possible statuses, visibility delay, consistency, and whether absence proves rejection or only no visible record.
Definitive non-application	Evidence that the original request cannot later apply, such as provider-confirmed rejection before execution. An empty search result is insufficient.
Compensation	Known effect identifier, allowed inverse action, authorization, idempotency, limitations, and verification of the inverse.
Dispatch limits	Rate limits, request deadlines, allowed retry horizon, resource scope, and side-effect classification.

Required connector profiles

- Payments: native idempotency plus lookup; exercise a provider retention boundary and a real sandbox adapter.
- CRM: no native key; lookup can be stale or incomplete. The normal ambiguous result remains unresolved unless authoritative terminal evidence exists.
- Email: irreversible action with no reliable idempotency or lookup. After an ambiguous first dispatch, stop automatic dispatch and escalate.

Connector contracts are versioned, backed by provider references where available, and attached to every decision. The test ledger is never exposed as a provider lookup endpoint.

04 Decision rules after an uncertain outcome

Evidence / capability	Allowed action
Valid native key; same payload; within documented scope and retention	Retry under the same operation and provider key, respecting concurrent-request and rate-limit behavior.
Provider confirms the intended effect was applied	Record the effect reference and evidence; mark confirmed success. Do not dispatch again.
Authoritative rejection or other evidence proving the original cannot later apply	A retry may be scheduled if policy permits. Preserve operation identity.
Lookup says not found, but can lag or the request may remain in progress	Keep UNKNOWN / VERIFYING. Recheck within a bounded investigation window, then escalate if still unresolved.
Native key expired or contract is unclear	Do not treat retry as protected. Reconcile or escalate.
Known applied effect requires an inverse; compensation is safe and authorized	Create a separately identified compensation action and verify its outcome.
No reliable evidence or safe repeat mechanism	Escalate. A human decision cannot manufacture proof of non-application.

The race the implementation must reproduce

Request A is accepted but delayed. The caller times out. Lookup returns not found. A naive recovery sends B. A and B later commit. SafeAct must either reuse provider-enforced idempotency or avoid B and preserve uncertainty. Sleeping for verify_after_seconds alone is not a correctness mechanism.

Error classification

Classification is connector-specific. A status code alone is not proof: some 4xx responses are transient, and a proxy error may follow a committed effect. Treat a transport failure as definitely not applied only when the transport path and connector establish that no dispatch occurred. Default ambiguous cases to UNKNOWN.

The investigation deadline controls when an operator is notified. It does not turn absence into proof or guarantee that a remote operation has stopped.

05 Durable state, concurrency, and architecture

```
RECORDED -> CLAIMED -> DISPATCHING
DISPATCHING -> SUCCEEDED | FAILED_CONFIRMED | UNKNOWN
UNKNOWN -> VERIFYING -> SUCCEEDED | RETRY_ELIGIBLE
                     -> UNKNOWN | ESCALATED_UNRESOLVED
RETRY_ELIGIBLE -> CLAIMED
Known applied effect -> compensation action -> verified outcome
```

Persist the action, canonical payload hash, audit event, and outbox entry in one PostgreSQL transaction before any network call. Workers claim eligible rows using transactional locking and an ownership version. Every state update verifies the expected state and owner version and appends an audit event in the same transaction.

- An expired local lease can trigger investigation. It is not permission to duplicate a remote dispatch.
- A stale worker may still complete its network call. Its response must be recorded as attempt evidence and reconciled without overwriting a newer decision.
- Local fencing protects database transitions. Remote fencing exists only if the provider participates; document the difference.
- Use database constraints plus a protected transition function or trigger to enforce legal transitions. A CHECK on the current state alone cannot validate the previous state.

Processing path

API -> PostgreSQL intent/outbox -> relay -> Celery worker -> connector -> provider. Verification workers and a reconciler inspect uncertain or expired attempts. The API and React application read durable evidence from PostgreSQL. Redis supports the broker, rate limits, and optional caches; its loss must not erase the operation record.

Delivery and commit uncertainty

The outbox relay may publish twice; workers must tolerate that. Specify broker acknowledgements, redelivery, worker-loss settings, and Redis persistence. After a database connection drops during commit, query by operation identity before repeating writes. Do not equate an exception with a rolled-back transaction.

A fixed two-step scenario may resume after a confirmed success or issue a verified compensation. Track each forward step and inverse separately. An inverse effect does not erase the original history.

06 Agent behavior, MCP, and approval boundaries

The agent is a bounded investigator and caller. It can inspect operation status, request allowed evidence collection, explain options, and propose an eligible next action. The policy engine makes the authoritative dispatch decision. The model cannot declare a lookup definitive, change connector contracts, or treat its own explanation as evidence.

Tool / actor	Permission boundary
submit_action / agent or service	Only an authorized action type and target; server validates tenant, operation identity, and policy.
get_action_status / inspect_evidence	Read only operations within the authenticated principal's scope. Treat provider text as untrusted input.
request_verification / propose_resolution	May enqueue bounded investigation or a proposal; cannot force a retry that policy disallows.
approve_resolution / human reviewer only	Separate authenticated endpoint. Bind approval to operation version, payload, evidence, expiry, and permitted next action.

Required agent evaluation

- Run at least one real MCP client with a real tool-calling model. Pin model identity, prompts, tool schemas, budgets, and scenario versions where the provider permits.
- Evaluate repeated trials on held-out scenario combinations: timeout after success, pending provider work, stale lookup, key expiry, denied access, malicious tool text, and attempted self-approval.
- Compare the same agent with direct tools versus guarded tools, and include a deterministic caller using the same safety layer. Report both the guard's contribution and the agent's contribution.
- Measure correct business outcome, justified escalation, unsafe attempts blocked, unauthorized approval attempts, tool calls, latency, and model cost. Publish failures; do not require an agent to outperform deterministic logic.

> **Human review may resolve evidence or authorize a documented safe action. An override that knowingly risks a duplicate is a different, explicitly labeled operation and cannot count as preserving SafeAct's safety guarantee.**

MCP is an interoperability interface, not proof of agent capability or an authorization system. Enforce identity and permissions at the service boundary [S3].

07 Full-stack product requirements

The React + TypeScript application is a required product surface, not an optional timeline screenshot. Deliver three complete user journeys with working backend enforcement.

Journey	Required behavior
Submit and follow an action	Validate inputs, submit once using stable operation identity, reconnect to live progress, and recover the view after a refresh.
Investigate uncertainty	Filter actions; inspect attempts, provider references, timestamps, capability assumptions, and the distinction between confirmed and unknown outcomes.
Review a proposal	Show the proposed change and supporting evidence, approve or reject an allowed action, reject stale approvals, and retain an audit record.

Access and usability

- Use established authentication/session libraries and seeded local demo accounts. The server derives tenant scope from the principal; a request body cannot choose another tenant.
- Define requester, investigator, and reviewer roles. Agents and service keys have separate scopes; API keys alone do not identify a human reviewer.
- Provide keyboard navigation, labeled forms, visible focus, readable contrast, responsive layouts, and loading, empty, disconnected, and error states.
- Use SSE or an equivalent update channel with reconnect and durable catch-up. The database remains the source of truth when events are missed.
- Handle two reviewers, repeated clicks, stale browser state, expired evidence, cancellation requests, and failed API calls. A cancellation request cannot imply an in-flight remote action was undone.

Meaningful frontend verification

Test the three journeys end to end, plus denied tenant access, stale approval rejection, duplicate submission, and connection interruption. Test actual outcomes and backend state rather than implementation details. A UI-only hidden button is not permission enforcement.

Data handling

Show synthetic or sandbox records in the public demo. Redact credentials and provider secrets from logs and evidence. Record evidence provenance and the actor responsible for each decision. Do not claim healthcare compliance, production security certification, or external adoption from these controls.

08 Fault catalog and independent ground truth

ID	Fault	Required observation
F01	Effect commits; response is lost	One applied effect; evidence-based success or unresolved status.
F02	Request rejected before execution	Retry only on definitive connector evidence.
F03	5xx after commit	UNKNOWN, then verification; no status-code shortcut.
F04	Duplicate delivery / concurrent submission	One local operation; provider protections respected.
F05	Worker dies after dispatch	Reconciler investigates before any redispatch.
F06	Lease expires while original request runs	No unprotected second dispatch; late completion handled.
F07	Lookup empty before late commit	Empty result does not become confirmed rejection.
F08	Redis lost / broker redelivery	Durable intent survives; coordination recovers.
F09	Idempotency retention expires	Replay is blocked or reconciled under a different contract.
F10	Same key, different payload	Conflict; no accidental reuse of a prior outcome.
F11	Database commit acknowledgement lost	Recover by operation identity; no blind reinsertion.
F12	Provider lookup fails or lies outside contract	UNKNOWN and escalation, not invented success.
F13	Compensation times out / fails	Separate uncertain compensation; no false restoration claim.
F14	Stale or concurrent approval	One valid version-bound decision.
F15	Agent changes key / attempts self-approval	Identity and permission checks reject bypass.
F16	429, long outage, or retry storm	Bounded backoff, limits, and unresolved evidence retained.

Mocks maintain an append-only effects ledger available only to the evaluation harness. Connector lookups expose only the information their declared provider profile permits. Use Toxiproxy for network behavior and explicit failpoints for commit/acknowledgement boundaries; network delays alone cannot select every critical execution point.

09 Evaluation, baselines, and reporting

Baseline	Purpose
B0: bounded naive retry	Demonstrate an unguarded failure mode. Label it as intentionally incomplete.
B1: queue redelivery / retry only	Separate durable execution from external effect verification.
B2: native provider idempotency	A legitimate baseline where the provider contract applies. It may match SafeAct.
B3: standard robust implementation	Durable intent/outbox, native keys, conservative reconciliation, and manual escalation; same connector capabilities.
SafeAct	Measure the reusable contracts, evidence workflow, guarded agent interface, and review ergonomics.

Experiment contract

Freeze connector profiles, fault timing, scenario families, payloads, run budget, and scoring rules before the final comparison. Use the same scenario seeds and controllable failpoints. Log actual interleavings: a fixed random seed does not guarantee identical operating-system scheduling. Include a no-fault control and adverse combinations not used during development.

Report distinct outcomes

- Applied effects per authorized operation and step; duplicate forward effects; orphan effects; and compensation effects, reported separately.
- Confirmed success, confirmed permanent failure, verified compensation, escalated unresolved, and still-pending actions. No accepted intent may disappear from accounting.
- Time to confirmed resolution and time to escalation as different metrics; queue wait, verification delay, and evidence coverage.
- Safety versus usefulness: a system that escalates everything must not appear equivalent to one that safely completes work.
- Throughput, tail latency, database contention, model/tool cost, and sandbox/provider limitations.

Minimum evidence target

Run at least 1,000 seeded scenarios across F01-F16, with documented coverage of critical interleavings and adversarial cases. The release target is zero observed prohibited duplicate effects within the declared contracts. Publish observed counts and coverage, not a universal probability claim. Any violation blocks qualification until investigated and rerun with the fix.

B0 or B1 need not fail every scenario. A robust baseline matching SafeAct is a legitimate result; report additional usability or integration value only if separately measured.

10 Implementation stack and operating environments

Component	Choice and purpose
API and validation	Python, FastAPI, Pydantic; pin supported versions at implementation start.
Durable state	PostgreSQL, SQLAlchemy, Alembic; unique identities, transactions, audit, and outbox.
Background work	Celery with Redis; explicit worker-loss, acknowledgement, retry, and persistence settings.
Agent interface	Official MCP SDK and one tool-calling model; separate service authorization.
Product UI	React, TypeScript, Vite; authenticated review and live investigation.
Faults and verification	pytest, Hypothesis, container integration tests, Toxiproxy, deterministic failpoints, independent ledger.
Observability	Structured logs, Prometheus metrics, optional OpenTelemetry traces.
Packaging and delivery	Docker Compose and GitHub Actions; documented fresh-clone execution.

Technology selection rule

Kubernetes, Terraform, and GPU serving are not required for SafeAct's core correctness. Use RolloutGate for that infrastructure evidence. Add a SafeAct deployment extension only if it answers a concrete operational question; preserve the reproducible Compose workflow.

Repository boundaries

```
core/           contracts, policies, state transitions
api/ workers/   authenticated endpoints and durable jobs
connectors/     mocks and sandbox provider adapter
mcp_server/     scoped tools and agent runner
ui/             React review application
chaos/ tests/    failpoints, schedules, baselines, scoring
docs/ deploy/   evidence, limits, local deployment
```

Start locally with synthetic data. Model calls and provider sandboxes may have usage charges; record actual costs. Do not describe CI, cloud services, or a model API as universally free. No production availability or operating history is implied by a local demonstration.

11 Implementation phases and acceptance gates

Phase	Work and exit evidence
A. Contract spike	Write provider assumptions; demonstrate the late-commit race; specify identities, outcomes, and evidence. Exit: independently reviewed decision table and failing race test.
B. Durable execution	Database model, transition enforcement, outbox, worker ownership, and payments mock. Exit: happy path plus crash/commit uncertainty behavior.
C. Verification and bounded repair	All connector profiles, investigation deadlines, fixed two-step workflow, safe compensation. Exit: F01-F13 exercise correct outcomes.
D. Product and agent	Auth, review console, permissions, MCP client, agent experiments, sandbox adapter. Exit: complete journeys and F14-F16.
E. Qualification	Freeze scenarios, compare B0-B3, run fault suite, load test, and fresh-clone validation. Exit: reproducible evidence and disclosed limits.

Required acceptance checks

- A1: no network dispatch before durable intent. A2: illegal/stale transitions and changed-payload key reuse are rejected.
- A3: ambiguous non-idempotent work never receives an unsupported automatic retry. A4: no forbidden duplicate effects in the declared qualification suite.
- A5: every accepted intent remains accounted for; unresolved escalation is distinct from completed work. A6: compensation history and uncertainty remain visible.
- A7: tenant isolation and human-only approval are tested at API and MCP boundaries. A8: three full-stack journeys pass under denied access, stale state, and reconnection.
- A9: real MCP-client trials and a real sandbox adapter have reproducible evidence. A10: baseline comparisons, failure seeds, actual interleavings, and costs are published.
- A11: CI runs focused regression and integration checks; the larger qualification suite has a separate reproducible command. A12: fresh-clone setup, architecture limits, and demo are verified.

Estimate effort only after phases A and B expose implementation uncertainty. Do not convert phase labels into a fixed six-week promise. Required correctness and authorization checks cannot be waived while retaining the corresponding claims.

12 Demonstration, evidence pack, and resume claims

Three-minute demonstration

- Show one refund operation and the provider's timeout-after-success fault. The timeline identifies the uncertainty and shows verified completion without a second dispatch.
- Show the harder late-commit / empty-lookup case. Explain why SafeAct remains unresolved instead of guessing.
- Attempt agent self-approval and a stale human approval. Both are rejected. A permitted human evidence-resolution path succeeds.
- Show the comparison report, including the robust baseline, unresolved rates, and known limits.

Required evidence pack

Include the architecture and connector contracts; schema and state-machine definitions; fault catalog; versioned test scenarios; baseline implementation details; raw outcomes; a reproduction command; agent prompts/tool versions; sandbox results; performance measurements; security boundary tests; and a concise failure analysis.

Result field	What to publish
Safety outcomes	Measured duplicate effects and orphan effects by scenario and connector.
Business completion	Measured successes, confirmed failures, verified compensations, unresolved escalations, and pending actions.
Recovery performance	Measured distributions for resolution and escalation; hardware and workload recorded.
Agent / product value	Measured tool behavior and usability observations; do not invent users, time savings, or adoption.

Resume wording after completion

Built a durable external-action service with PostgreSQL, Celery, and Redis; evaluated [N] fault-injected scenarios with [measured outcomes], including late provider completion and worker crashes.

Implemented scoped MCP tools and a React review application with version-bound human approval; measured [agent results] across [scenario coverage].

Replace bracketed fields only after measurement. Do not claim exactly-once behavior across arbitrary APIs, production operation, a published benchmark win, or guaranteed hiring outcomes.

13 References and final qualification checklist

Primary references checked during the October 15, 2026 review. They establish existing behavior and prior art. They do not validate an unbuilt SafeAct implementation.

[S1] **AWS Builders' Library - Making retries safe with idempotent APIs**

https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

Stable request identity, parameter mismatch, late requests, and retention limits.

[S2] **Stripe - Webhook delivery behavior and best practices**

https://docs.stripe.com/webhooks

Duplicates, event ordering, versioned payloads, and asynchronous processing. Pin the provider contract used by the sandbox adapter.

[S3] **Model Context Protocol - Architecture**

https://modelcontextprotocol.io/docs/learn/architecture

Protocol concepts and client/server responsibilities. Service permissions remain application requirements.

[S4] **Celery - Tasks**

https://docs.celeryq.dev/en/stable/userguide/tasks.html

Recheck acknowledgement, worker-loss, and retry configuration against the pinned release before implementation.

Definition of done

- All required acceptance checks have linked evidence. Any failing check narrows the published claim or blocks release.
- Mocks and real sandbox behavior are identified separately. The independent ledger cannot leak into connector logic.
- Results contain observed values with workload, versions, and reproduction instructions.
- The application is usable from a fresh clone; the reviewer can inspect uncertainty and denied actions.
- The README states connector assumptions, unsupported faults, unresolved cases, and the difference between local coordination and remote guarantees.

> **Final claim: a rigorously evaluated implementation of bounded, evidence-based action execution. Success comes from the correctness and clarity of the evidence, not the number of technologies included.**